Lab 04 — The Dockerfile: building your own images

Theory — the recipe, the context, the cache, the two traps that cost the most, and what a build engine without a daemon changes.

Objectives

- Understand what the **build context** is and why it determines what you can copy.
- Know the essential instructions and what each produces.
- Tell `CMD` from `ENTRYPOINT`, *shell* form from *exec* form.
- Tell `ARG` from `ENV`.
- Order a Dockerfile to exploit the **build cache**.

---

1. The build context

```
podman build -t my-api:1.0 .
```

The trailing `.` is not decorative: it is the **build context**, the folder the build is allowed to read. Two absolute consequences:

- **You can only copy what is in the context.** `COPY ../secrets/key.pem .` always fails: the file is outside the perimeter. No workaround.
- **The whole folder is part of the context**, including `.git/`, `node_modules/`, `target/`, logs and local configuration. A `COPY . .` puts all of it into the image.

> **PODMAN**
> With Docker, the client **packages** the context into an archive and **sends** it to the daemon — that is the `transferring context: 900MB` line that makes Angular builds drag on. With Podman, the build is done by **Buildah**, integrated into the same process, which reads the folder directly: no archive, no transfer. The context remains the boundary of what is copyable, and `.dockerignore` remains indispensable — not for speed, but for what ends up **in the image**. Buildah also accepts the neutral names `Containerfile` and `.containerignore`.

The **`.dockerignore`** file, at the root of the context, excludes what must not enter:

```
.git
node_modules
target
.env
```

> **PITFALL**
> Without `.dockerignore`, a `COPY . .` puts local secrets and the whole `.git` **into the final image**, for anyone who downloads it. A classic data leak.

The Dockerfile itself may live elsewhere: `-f docker/api.Dockerfile` designates it.

2. The essential instructions

```
FROM docker.io/library/eclipse-temurin:21-jre-alpine   # base image — always the 1st instruction
LABEL org.opencontainers.image.source="https://git.mycompany.be/payments/api"
WORKDIR /app                                  # creates and enters the folder
COPY target/api.jar /app/api.jar              # copies from the context into the image
ENV JAVA_OPTS="-XX:MaxRAMPercentage=75"       # variable present at run time
EXPOSE 8080                                   # documentation: publishes NO port
USER 1000:1000                                # do not run as root
ENTRYPOINT ["sh","-c","exec java $JAVA_OPTS -jar /app/api.jar"]
```

Instruction	Role	File layer?
`FROM`	Starting point	yes (the base's)
`RUN`	Runs a command **at build time**	yes
`COPY` / `ADD`	Copies from the context	yes
`WORKDIR`, `ENV`, `USER`, `EXPOSE`, `LABEL`	Metadata	no (`0B`)
`CMD`, `ENTRYPOINT`	What runs at `run`	no
`ARG`	Variable **for the build only**	no

Three clarifications. **`COPY` rather than `ADD`**: `ADD` unpacks archives and downloads URLs, two implicit behaviours. **`EXPOSE` publishes nothing**: `-p` publishes (lab 07). **`RUN` runs at build time**: `RUN java -jar api.jar` would launch the application during construction.

> **REMEMBER**
> Write the `FROM` in full: `docker.io/library/eclipse-temurin:21-jre-alpine`. A `FROM eclipse-temurin:…` depends on the configuration of the building machine (lab 02); in a company it will be `registry.internal/base/…`.

3. `CMD` versus `ENTRYPOINT`

Both define what runs at start-up. Their difference is their relationship to the arguments of `podman run`:

- **`CMD`** is a **default, replaceable** value. `podman run my-image other-command` ignores the `CMD`.
- **`ENTRYPOINT`** is the **fixed** program. The arguments of `podman run` are **appended** to it.

```
ENTRYPOINT ["java","-jar","/app/api.jar"]
CMD ["--spring.profiles.active=prod"]
```

`podman run api` runs `java -jar /app/api.jar --spring.profiles.active=prod`; `podman run api --spring.profiles.active=dev` runs the same with the `dev` profile. That is the standard pattern: `ENTRYPOINT` fixes the program, `CMD` provides the default arguments; `podman run --entrypoint sh -it my-image` remains the escape hatch for debugging.

→ SPRING BOOT
Arguments passed after the JAR (`--spring.profiles.active=dev` , `--server.port=9090`) are read by Spring as properties that take precedence over `application.yml` . Hence the convenience of the `ENTRYPOINT` + `CMD` duo: same image, a different argument per environment. Lab 08 will show that environment variables are even better.

4. *Shell* form and *exec* form

Any command can be written in two ways, and it is not a matter of style:

```
CMD java -jar /app/api.jar                # SHELL form  -> /bin/sh -c "java -jar ..."
CMD ["java","-jar","/app/api.jar"]        # EXEC form   -> java becomes PID 1
```

In *exec* form, the application **is** PID 1: it receives `SIGTERM` and stops cleanly. In *shell* form, a `/bin/sh` sits in between — the problem of lab 03: a shell that stays PID 1 does not forward `SIGTERM` , the application never receives it, `stop` waits ten seconds and kills everything.

The important word is **stays**. The behaviour depends on the shell implementation:

Case	PID 1	`podman stop`
Exec form	your application	clean, code 143
Shell form, simple command, **Alpine** base (busybox)	your application (the shell steps aside)	clean, code 143
Shell form, simple command, **Debian/Ubuntu** base (dash)	`/bin/sh`	10 s then code 137
Shell form with a pipe, an `&` , a `;`	`/bin/sh`	10 s then code 137
Entry script that launches the app **without** `exec`	`/bin/sh`	10 s then code 137

→ LINUX / SHELL
`/bin/sh` is not a single program: `dash` on Debian and Ubuntu, busybox's `ash` on Alpine. Some, when the command is the *last* of the script, replace themselves with it (an implicit `exec`); others create a child and wait. Hence a Dockerfile that stops cleanly on Alpine and not on Debian.

REMEMBER
Always write the *exec* form, with JSON double quotes. If you must go through a shell, write it explicitly **and** use `exec` : `ENTRYPOINT ["sh","-c","exec java $JAVA_OPTS -jar /app/api.jar"]` .

A lesser-known consequence: in *exec* form, `$JAVA_OPTS` , `&&` , `|` and `>` are **not** interpreted — there is no shell to do it.

5. `ARG` versus `ENV`

```
ARG VERSION=1.0            # available during the build only
ENV APP_VERSION=${VERSION} # persists in the image and in the containers
```

	`ARG`	`ENV`
Visible during the build	yes	yes
Present in the final image	**no**	yes
Changeable at build time	`--build-arg VERSION=2.0`	no
Changeable at run time	no	`podman run -e APP_VERSION=…`

PITFALL
"`ARG` disappears from the image" does **not** mean "`ARG` is safe for a secret": the value remains visible in `podman history` and in the build cache. A password passed with `--build-arg` is a leak (lab 08).

6. The build cache

The engine processes instructions in order and caches every result. For each one, it asks: "have I already run this one, starting from the same previous layer?" If yes, it reuses (`--> Using cache`). Otherwise it runs it — **and invalidates everything that follows**. Invalidation comes from a change in the instruction text, in the **content** of copied files (`COPY` / `ADD`), or from an invalidated previous instruction, in cascade. Hence the golden rule: **from most stable to most volatile**.

```
# BAD: the code changes at every commit, so everything is rebuilt
COPY . /app
RUN mvn dependency:go-offline

# GOOD: dependencies are only re-downloaded if pom.xml changes
COPY pom.xml /app/
RUN mvn dependency:go-offline
COPY src /app/src
```

Same reasoning for Angular: `COPY package*.json` then `npm ci`, and only then `COPY . .`. The gain counts in minutes per CI build — and, since an unchanged layer is not retransferred at `push` (lab 02), in deployment time. To rebuild everything: `--no-cache`.

7. Writing a correct `RUN`

```
# BAD: three layers, a 40 MB apt cache shipped in the image
RUN apt-get update
RUN apt-get install -y curl
RUN rm -rf /var/lib/apt/lists/*

# GOOD: a single layer, effective clean-up
RUN apt-get update \
 && apt-get install -y --no-install-recommends curl \
 && rm -rf /var/lib/apt/lists/*
```

Clean-up must happen **in the same** `RUN` as the installation: a later deletion hides without removing (lab 02). And `apt-get update` must never be alone in its layer: cached for weeks, it would serve stale indexes — the *cache busting* problem.

8. In the workplace

The Dockerfile of a Spring Boot back end looks like this (simple version; the multi-stage version comes in lab 05):

```
FROM registry.internal/base/eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY target/api-*.jar app.jar
EXPOSE 8080
USER 1000:1000
ENTRYPOINT ["java","-jar","/app/app.jar"]
```

Points to note: a **JRE** image, not a JDK, pulled from the internal registry, non-root `USER`, *exec* form, JAR copied from `target/` — so the Maven build happened **before**, in CI; that is the limit multi-stage will lift. On the Angular side, you never containerise `ng serve`, but the result of `ng build` served by nginx. The same recipe is built by Docker in CI and by Podman on your workstation: a Dockerfile is a standard.

Remember

- The `.` of `build .` designates the **context**: what is copyable, and what a `COPY . .` ships. `.dockerignore` is mandatory — even without a transfer, with Podman.
- `EXPOSE` documents, `-p` publishes. `RUN` runs at build time, `CMD` / `ENTRYPOINT` at run time; `ENTRYPOINT` fixes the program, `CMD` provides replaceable arguments.
- *Exec* form `["prog","arg"]`: the application is PID 1 and receives `SIGTERM`. *Shell* form: it depends on the base image's shell — so no. `ARG` lives during the build, `ENV` persists in the image — neither is suitable for a secret.
- Order from most stable to most volatile; any invalidated instruction invalidates the following ones.
- Installation and clean-up in the **same** `RUN`, otherwise the files remain.

Vocabulary

build context: folder the build is allowed to read. — **`.dockerignore` / `.containerignore`**: context exclusions. — **Containerfile**: Podman's neutral name for the Dockerfile. — **Buildah**: Podman's build engine. — **exec / shell form**: two ways of writing `CMD` / `ENTRYPOINT`. — **cache busting**: deliberate cache invalidation. — **base image**: the image named by `FROM`.